Data Analytics · Python Lab Exercise

Pandas Essentials — Class Practice 7.2

Jupyter Notebook · Pandas · DataFrame · Series · Data Cleaning · GroupBy · Filtering

1. Scenario

RetailHub Inc. — Employee Sales Performance Tracker

You are a data analyst at RetailHub Inc., a national retail chain with branches across five cities. Your manager has asked you to build a Jupyter Notebook that uses the Pandas library to load, clean, explore, and summarise employee sales data — producing actionable insights about team performance.

Your completed notebook must:

1. Create a Pandas DataFrame from a Python dictionary representing employee records.
2. Inspect the DataFrame using built-in Pandas attributes and methods.
3. Select, filter, and sort data using Pandas operations.
4. Add a calculated column and apply a conditional label using a function.
5. Group the data by city and compute aggregated statistics.
6. Document every step with a Markdown cell above each Code cell.

Jupyter Setup

Launch Jupyter: jupyter notebook in your terminal.
Create a new notebook — File > New > Notebook > Python 3 (ipykernel).
Rename it: DA_Exercise_7_2.ipynb
Run each Code cell with Shift+Enter. Add Markdown cells with Esc → M.
Save frequently with Ctrl+S.

2. Requirements

Your notebook must satisfy all six requirements below. Each requirement maps to one or more Code cells, each preceded by a Markdown cell.

1 Req

Notebook Header — Markdown Cell

Add a Markdown cell at the very top containing:

```
# Data Analytics — Exercise 7.2: Pandas Essentials
## Your Name | Date
A brief (2-sentence) description of what this notebook analyses.
```

2 Req	**Import Pandas and Create the DataFrame** import pandas as pd — always the first Code cell after the header. Create a DataFrame called df from this employee dictionary: 'name' : 8 employee names (strings) 'city' : 8 cities — mix of London, Manchester, Birmingham, Leeds 'sales' : 8 float values representing monthly sales in pounds 'months' : 8 int values representing months in the company (1–24) Print df, df.shape, df.dtypes, and df.info().
3 Req	**Inspect and Describe the Data** Use df.head(3) to print the first 3 rows. Use df.tail(3) to print the last 3 rows. Use df.describe() to print summary statistics for numeric columns. Use df['sales'].mean() and df['sales'].max() formatted to 2dp with f-strings.
4 Req	**Select, Filter, and Sort** Select and print only the 'name' and 'sales' columns as a new DataFrame. Filter and print all rows where sales > 3500.0 using Boolean indexing. Sort the full DataFrame by 'sales' descending and print the result. Use df.iloc[0] to print the top-selling employee's full record.
5 Req	**Add a Calculated Column and Conditional Label** Add a new column 'commission' = sales * 0.08 (8% commission on every sale). Add a new column 'rating' using a lambda with apply(): 'Star' if sales >= 4000 'Good' if sales >= 2500 'Developing' otherwise Print the updated DataFrame showing all five columns. Format commission values to 2 decimal places inside a for loop using f-strings.
6 Req	**GroupBy — Aggregate Statistics by City** Use df.groupby('city')['sales'].sum() to get total sales per city. Use df.groupby('city')['sales'].mean() to get mean sales per city. Use df.groupby('city').agg({'sales':['sum','mean','count']}) for a full summary. Use idxmax() on the grouped totals to find the top city by total sales. Write a Markdown Summary cell interpreting the GroupBy findings in 3 sentences.

3. Hints

Pandas quick reference — compare the given example vs your task

Given example (student scores)	Your task (employee sales)
`import pandas as pd`	`import pandas as pd`
`df = pd.DataFrame({'name':['Ana','Ben'],'score':[88,72]})`	`df = pd.DataFrame({'name':[...],'city':[...],...})`
`df.shape  # → (2, 2)`	`df.shape  # → (8, 4)`
`df.describe()  # stats for numeric cols`	`df.describe()  # stats for sales & months`
`df[df['score'] > 80]  # Boolean filter`	`df[df['sales'] > 3500]  # Boolean filter`
`df.sort_values('score', ascending=False)`	`df.sort_values('sales', ascending=False)`
`df['bonus'] = df['score'] * 0.05`	`df['commission'] = df['sales'] * 0.08`
`df.groupby('class')['score'].mean()`	`df.groupby('city')['sales'].sum()`

1 Hint

Import convention — always use pd as the alias

```
import pandas as pd
```

Using pd as the alias is the universal Pandas convention in data analytics.

```
DataFrames are created with pd.DataFrame() and Series with pd.Series().
Every Pandas method is accessed as df.method() or pd.function().
```

2 Hint

Creating a DataFrame from a dictionary

data = {'name': ['Alice', 'Bob', ...], 'sales': [4200.0, 3100.0, ...]}

```
df = pd.DataFrame(data)
```

Each dictionary key becomes a column name; each list becomes a column of values.
All lists must be the same length — 8 items each in this exercise.

3 Hint

Formatting with f-strings and iterating rows

Use :.2f inside curly braces for 2 decimal places:

```
  print(f'Mean sales: £{df["sales"].mean():,.2f}')
To iterate rows and print commission: use df.iterrows():
  for _, row in df.iterrows():
      print(f'{row["name"]}: £{row["commission"]:,.2f}')
```

4 Hint

Adding columns and using apply() with a lambda

df['commission'] = df['sales'] * 0.08 # element-wise — no loop needed
df['rating'] = df['sales'].apply(lambda x: 'Star' if x >= 4000

```
    else ('Good' if x >= 2500 else 'Developing'))
```

apply() passes each value in the column to the lambda function.

5

GroupBy — sum, mean, count, and finding the top group

Hint

```
  city_totals = df.groupby('city')['sales'].sum()
  top_city = city_totals.idxmax()   # returns the city name (index label)
  print(f'Top city: {top_city}  Total: £{city_totals[top_city]:,.2f}')
```

agg() lets you compute multiple statistics in one call:

```
  df.groupby('city').agg({'sales': ['sum','mean','count']})
```

4. Solution

> *Attempt all six requirements in your notebook before reading the solution below. Use the hints and the example table as your guide.*

DA_Exercise_7_2.ipynb — complete notebook cell sequence

CELL 1 — Markdown cell (Esc → M before typing)

Markdown — *Notebook header*

```
# Data Analytics — Exercise 7.2: Pandas Essentials
## Your Name  |  Date: 2025-01-22

This notebook uses the Pandas library to load, inspect, filter,
and summarise employee sales data for RetailHub Inc.
It covers DataFrame creation, Boolean filtering, calculated columns,
apply() with a lambda, and GroupBy aggregation.
```

CELL 2 — Markdown cell

Markdown — *Section heading — import and data*

```
## 1. Import Pandas and Create the DataFrame
We import Pandas and build a DataFrame from a dictionary of employee records.
```

CELL 3 — Code cell (Shift+Enter to run)

In [ ]: *Requirement 2 — import Pandas and create DataFrame*

```
# Requirement 1 — import Pandas with standard alias
import pandas as pd

# Requirement 2 — employee sales data (8 records, 4 columns)
data = {
    'name'    : ['Alice', 'Ben', 'Carmen', 'Dev',
                 'Eve', 'Frank', 'Grace', 'Hiro'],
    'city'    : ['London', 'Manchester', 'London', 'Leeds',
```

```
                'Birmingham', 'Manchester', 'Leeds', 'London'],
    'sales'   : [4200.0, 3100.0, 5050.0, 2800.0,
                 1950.0, 3750.0, 4600.0, 3300.0],
    'months'  : [12, 6, 24, 3, 18, 9, 15, 7]
}

df = pd.DataFrame(data)

print('── Employee DataFrame ──────────────────────')
print(df)
print(f'\nShape  : {df.shape}')
print(f'dtypes :\n{df.dtypes}')
print('\ndf.info():')
df.info()
```

Out [ ]: output

```
── Employee DataFrame ──────────────────────
    name        city   sales  months
0  Alice      London  4200.0      12
1    Ben  Manchester  3100.0       6
2  Carmen     London  5050.0      24
3    Dev       Leeds  2800.0       3
4    Eve  Birmingham  1950.0      18
5  Frank  Manchester  3750.0       9
6  Grace       Leeds  4600.0      15
7   Hiro      London  3300.0       7

Shape  : (8, 4)
dtypes :
name      object
city      object
sales    float64
months     int64
```

CELL 4 — Markdown cell

Markdown — *Section heading — inspect*

```
## 2. Inspect and Describe the Data
We use Pandas methods to preview the data and compute summary statistics.
```

CELL 5 — Code cell

In [ ]: *Requirement 3 — inspect and describe*

```
print('First 3 rows:\n', df.head(3))
print('\nLast 3 rows:\n', df.tail(3))
print('\nSummary statistics:\n', df.describe())

print(f'\nMean sales : £{df["sales"].mean():>10,.2f}')
print(f'Max  sales : £{df["sales"].max():>10,.2f}')
print(f'Min  sales : £{df["sales"].min():>10,.2f}')
print(f'Total sales: £{df["sales"].sum():>10,.2f}')
```

Out [ ]: output

```
First 3 rows:
     name        city   sales  months
0   Alice      London  4200.0      12
1     Ben  Manchester  3100.0       6
2  Carmen      London  5050.0      24

Last 3 rows:
   name        city   sales  months
5  Frank  Manchester  3750.0       9
6  Grace       Leeds  4600.0      15
7   Hiro      London  3300.0       7

Mean sales : £     3,593.75
Max  sales : £     5,050.00
Min  sales : £     1,950.00
Total sales: £    28,750.00
```

CELL 6 — Markdown cell

Markdown — *Section heading — select, filter, sort*

```
## 3. Select, Filter, and Sort
Pandas makes it easy to slice columns, filter rows, and sort results.
```

CELL 7 — Code cell

In [ ]: *Requirement 4 — select, filter, sort*

```
# Select two columns
print('Name & Sales columns:\n', df[['name', 'sales']])

# Boolean filter — sales above £3500
high_sales = df[df['sales'] > 3500.0]
print('\nSales > £3500:\n', high_sales)

# Sort by sales descending
df_sorted = df.sort_values('sales', ascending=False).reset_index(drop=True)
print('\nSorted by sales (highest first):\n', df_sorted)

# Top seller — first row after sort
print('\nTop seller record:\n', df_sorted.iloc[0])
```

Out [ ]: output

```
Name & Sales columns:
     name   sales
0   Alice  4200.0
1     Ben  3100.0  ...

Sales > £3500:
     name        city   sales  months
0   Alice      London  4200.0      12
2  Carmen      London  5050.0      24
```

```
5   Frank  Manchester  3750.0       9
6   Grace       Leeds  4600.0      15

Sorted by sales (highest first):
     name    city   sales  months
0  Carmen  London  5050.0      24
1   Grace   Leeds  4600.0      15  ...

Top seller record:
name      Carmen
city      London
sales     5050.0
months        24
```

CELL 8 — Markdown cell

Markdown — *Section heading — calculated columns*

```
## 4. Calculated Column and Conditional Rating
We add commission (8% of sales) and a performance rating using apply().
```

CELL 9 — Code cell

In [ ]: *Requirement 5 — commission and rating columns*

```
# Add commission column — element-wise (no loop needed)
df['commission'] = df['sales'] * 0.08

# Add rating using apply() with a lambda
df['rating'] = df['sales'].apply(
    lambda x: 'Star' if x >= 4000
    else ('Good' if x >= 2500 else 'Developing')
)

print('Updated DataFrame:\n', df)

# Print each employee's commission using f-string + iterrows()
print('\n── Commission per employee ──────────────')
for _, row in df.iterrows():
    print(f'{row["name"]:<8} | £{row["commission"]:>8,.2f} | {row["rating"]}')
```

Out [ ]: output

```
Updated DataFrame:
     name        city   sales  months  commission  rating
0   Alice      London  4200.0      12       336.0    Star
1     Ben  Manchester  3100.0       6       248.0    Good
2  Carmen      London  5050.0      24       404.0    Star
3     Dev       Leeds  2800.0       3       224.0    Good
4     Eve  Birmingham  1950.0      18       156.0    Developing
5   Frank  Manchester  3750.0       9       300.0    Good
6   Grace       Leeds  4600.0      15       368.0    Star
7    Hiro      London  3300.0       7       264.0    Good
```

```
── Commission per employee ──────────────
Alice    |    £336.00 | Star
Ben      |    £248.00 | Good
Carmen   |    £404.00 | Star
Dev      |    £224.00 | Good
Eve      |    £156.00 | Developing
Frank    |    £300.00 | Good
Grace    |    £368.00 | Star
Hiro     |    £264.00 | Good
```

CELL 10 — Markdown cell

Markdown — *Section heading — GroupBy*

```
## 5. GroupBy — Sales by City
GroupBy lets us aggregate data by a category, revealing city-level performance.
```

CELL 11 — Code cell

In [ ]: *Requirement 6 — GroupBy aggregation*

```python
# Total sales per city
city_totals = df.groupby('city')['sales'].sum()
print('── Total Sales by City ──────────────────')
print(city_totals)

# Mean sales per city
city_means = df.groupby('city')['sales'].mean()
print('\n── Mean Sales by City ───────────────────')
print(city_means)

# Full aggregation — sum, mean, count per city
city_agg = df.groupby('city').agg({'sales': ['sum', 'mean', 'count']})
print('\n── Full City Summary ────────────────────')
print(city_agg)

# Find the top city
top_city = city_totals.idxmax()
print(f'\nTop city: {top_city}  Total: £{city_totals[top_city]:,.2f}')
```

Out [ ]: output

```
── Total Sales by City ──────────────────
city
Birmingham     1950.0
Leeds          7400.0
London        12550.0
Manchester     6850.0

── Mean Sales by City ───────────────────
city
Birmingham    1950.000000
Leeds         3700.000000
London        4183.333333
```

```
Manchester    3425.000000

── Full City Summary ──────────────────────
              sales
               sum     mean  count
city
Birmingham    1950.0  1950.0      1
Leeds         7400.0  3700.0      2
London       12550.0  4183.3      3
Manchester    6850.0  3425.0      2

Top city: London  Total: £12,550.00
```

CELL 12 — Markdown cell

➡ **Markdown** — *Summary interpretation*

```
## Summary

- **London** is the top-performing city with a combined total of **£12,550**
from 3 employees.
- **3 of 8 employees** achieved Star rating (sales ≥ £4,000); only Eve
(Birmingham) is Developing.
- The team's average monthly sales of **£3,593.75** and top performer Carmen
(£5,050) suggest
  London should be prioritised for investment in the next quarter.
```

5. Expected Output

With the employee data defined above, running Kernel > Restart & Run All produces the following output across all Code cells:

Notebook Output

```
── Cell 3 ──────────────────────────────────────────────────
    name        city   sales  months
0  Alice      London  4200.0      12
1    Ben  Manchester  3100.0       6
2  Carmen     London  5050.0      24
3    Dev       Leeds  2800.0       3
4    Eve  Birmingham  1950.0      18
5  Frank  Manchester  3750.0       9
6  Grace       Leeds  4600.0      15
7   Hiro      London  3300.0       7
Shape  : (8, 4)   dtype: sales=float64, months=int64

── Cell 5 ──────────────────────────────────────────────────
Mean sales : £     3,593.75
Max  sales : £     5,050.00
```

```
Min  sales : £     1,950.00
Total sales: £    28,750.00

── Cell 7 ──────────────────────────────────────────────────
4 employees have sales > £3,500  (Alice, Carmen, Frank, Grace)
Sorted top seller: Carmen | London | £5,050.00

── Cell 9 ──────────────────────────────────────────────────
Alice    |    £336.00 | Star
Ben      |    £248.00 | Good
Carmen   |    £404.00 | Star
Dev      |    £224.00 | Good
Eve      |    £156.00 | Developing
Frank    |    £300.00 | Good
Grace    |    £368.00 | Star
Hiro     |    £264.00 | Good

── Cell 11 ─────────────────────────────────────────────────
London        £12,550.00  (3 employees)
Leeds          £7,400.00  (2 employees)
Manchester     £6,850.00  (2 employees)
Birmingham     £1,950.00  (1 employee)
Top city: London  Total: £12,550.00
```

Cell breakdown — what each cell produces

Cell	Type	Requirement	Key Output
1	Markdown	Header	Formatted notebook title and description
2	Markdown	Intro	Section heading for import/DataFrame
3	Code	Req 2	DataFrame printed, shape, dtypes, info()
4	Markdown	Intro	Section heading for inspect/describe
5	Code	Req 3	head, tail, describe, mean/max/min/sum to 2dp
6	Markdown	Intro	Section heading for filter/sort
7	Code	Req 4	Column slice, Boolean filter, sorted df, iloc[0]
8	Markdown	Intro	Section heading for calculated columns
9	Code	Req 5	commission + rating columns, iterrows f-string loop
10	Markdown	Intro	Section heading for GroupBy
11	Code	Req 6	city_totals, city_means, city_agg, top city

12	Markdown	Summary	3-sentence written interpretation

6. Pandas Quick Reference

The table below maps every Pandas concept used in this exercise to its syntax and purpose.

Concept	Syntax used in this exercise	What it does
Create DataFrame	`pd.DataFrame(dict)`	Converts a dictionary of equal-length lists to a table
Shape	`df.shape`	Returns a tuple (rows, cols) e.g. (8, 4)
Data types	`df.dtypes`	Shows the dtype of each column
Info summary	`df.info()`	Prints shape, dtypes, and non-null counts together
Preview top rows	`df.head(n)`	Returns first n rows (default 5)
Preview bottom rows	`df.tail(n)`	Returns last n rows (default 5)
Descriptive stats	`df.describe()`	Count, mean, std, min, quartiles, max
Column stat	`df['col'].mean()`	Computes mean of one column; also .sum() .max() .min()
Select columns	`df[['col1','col2']]`	Returns a new DataFrame with only

		those columns
Boolean filter	`df[df['col'] > val]`	Returns rows where condition is True
Sort rows	`df.sort_values('col', ascending=False)`	Sorts DataFrame by column; descending with False
Row by position	`df.iloc[0]`	Returns the row at integer position 0 (top row)
New column	`df['col'] = df['other'] * n`	Adds a calculated column element-wise
apply() lambda	`df['col'].apply(lambda x: ...)`	Applies a function to every value in a column
Iterate rows	`df.iterrows()`	Yields (index, Series) pairs for each row
GroupBy total	`df.groupby('col')['val'].sum()`	Sums val for each unique group in col
GroupBy mean	`df.groupby('col')['val'].mean()`	Mean of val for each group
GroupBy multi-stat	`df.groupby('col').agg({'val':['sum','mean','count']})`	Multiple stats in one call
Top group index	`series.idxmax()`	Returns the index label of the maximum value

7. Extension Challenges

Once your base notebook runs correctly with Kernel > Restart & Run All, try these extensions:

- Add a Code cell that uses df['sales'].value_counts(bins=3) to bin the sales column into three equal-width bands and count how many employees fall into each. Print the result with a descriptive label.
- Add a Code cell that creates a pivot table using pd.pivot_table(df, values='sales', index='city', aggfunc='mean'). Print the pivot and compare it to the GroupBy mean output from Cell 11.
- Add a Code cell that uses df.sort_values('months').reset_index(drop=True) to rank employees by tenure. Add a 'rank' column using df.index + 1 and print the ranked table.
- Add a Code cell that uses df[df['rating'] == 'Star'][['name','city','sales']] to create a Star performers DataFrame. Print it and calculate their combined commission using .sum().
- Simulate missing data: manually set df.loc[2, 'sales'] = None, then use df.isnull().sum() to detect it, df['sales'].fillna(df['sales'].mean()) to fill it, and print the cleaned DataFrame.
- Add a Code cell that exports the final DataFrame to a CSV: df.to_csv('employee_sales.csv', index=False). Then re-read it with pd.read_csv('employee_sales.csv') and verify the shape matches.